

Lekcja 09: Breakout

Cel

Arkanoid: piłka, paletka i 40 cegiełek do zbitia. Łączysz dwie ostatnie lekcje: **tablica struktur**.

Co nowego

W Flappy druga rura wymagała skopiowania połowy kodu. Rozwiązanie: tablica, której każdy element jest strukturą.

```
struct Brick { int x; int y; bool alive; Color color; };
Brick bricks[40]; // 40 cegiełek, każda z czterema polami

bricks[7].alive = false; // pole struktury o indeksie 7
for (int i = 0; i < 40; i++) { // petla po wszystkich
    if (bricks[i].alive) rect(bricks[i].x, bricks[i].y, BRICK_W, BRICK_H, bricks[i].color);
}
```

Siatka w jednowymiarowej tablicy: numer cegiełki `i = row * BRICK_COLS + col`. Z numeru wraca się `row = i / BRICK_COLS`, `col = i % BRICK_COLS` – dzielenie całkowite i reszta.

To ostatni „klocek” składni potrzebny do dowolnej gry 2D: struktury opisują rzeczy, tablice trzymają ich wiele, pętle je przetwarzają, `if` decyduje. Reszta to pomysły.

Uruchom

Otwórz `gra.cpp`, **Ctrl+Shift+B**. Cztery kolorowe rzędy cegiełek, biała paletka (strzałki), piłka startuje z paletki. Piłka odbija się od ścian, sufitu i paletki, ale **przelatuje przez cegiełki** – to zadanie 2. Spadnięcie piłki odejmuje życie (czerwona liczba w prawym rogu).

Jak to działa

`setup()` wypełnia tablicę w podwójnej pętli: pozycja z numeru kolumny i rzędu, kolor z rzędu. `frame()` rusza paletką i piłką (struktura `Ball`), odbija ją, a na końcu **jedną pętlą** rysuje wszystkie żywe cegiełki.

Zadania

- ★ Zmień kolory rzędów i rozmiar cegiełek. Dodaj piąty rząd – co trzeba zmienić? (Tylko `BRICK_ROWS` i jeden `if` koloru.)
- ★★ **Zbijanie.** Pętla po cegiełkach: jeśli `bricks[i].alive` i `overlaps(...)` piłki z cegiełką – `alive = false`, `ball.vy = -ball.vy`, `score += 10`. Żeby piłka nie zbiła dwóch cegiełek naraz, po trafieniu przerwij pętlę słowem `break`; . (test 1)
- ★★ Koniec gry: funkcja `int bricks_left()` liczy żywe; gdy 0 – „WYGRALES”; gdy `lives == 0` – „GAME OVER”. Oba: `restart()` po A.

4. ★★ Kąt odbicia od paletki zależy od miejsca trafienia: `ball.vx = (ball.x - (paddle_x + PADDLE_W / 2)) / 8;` (środek paletki – prosto w górę, brzegi – na ukos). Pilnuj, żeby `vx` nie było 0 przez przypadek... albo niech będzie – to ciekawy przypadek.
5. ★★★ Cegielki twarde: dodaj do `Brick` pole `int hits;` (ile trafień zostało). Rząd 0 wymaga 2 trafień i po pierwszym ciemnieje (`color = DARK_RED`). Do tego bonus: co dziesiąta zbita cegielka wydłuża paletkę o 20 px (`PADDLE_W` na zmienną).

Sprawdź sam

Po zadaniu 2 piłka startuje z paletki w górę i po chwili zbija pierwszą cegielkę – bez dotykania klawiszy:

```
console_sim.exe --game breakout --frames 301 --trace 60
```

W ostatniej linii `score` większe od 0.

Dla ciekawych

`break` wychodzi z pętli natychmiast. Jest też `continue` – przeskakuje do następnego obrotu. Spróbuj napisać pętlę rysowania z `continue` zamiast `if (alive): if (!bricks[i].alive) continue;`.

Słowniczek

po polsku	w C++
tablica struktur	<code>Brick bricks[40];</code>
pole elementu tablicy	<code>bricks[i].x</code>
numer z rzędu i kolumny	<code>i = row * COLS + col</code>
przerwij pętlę	<code>break;</code>
następny obrót pętli	<code>continue;</code>